

NanoInfer 多模型架构适配设计

NanoInfer

2026-08-20

目标：从复现 *Qwen2.5* 扩展到支持多种 LLM 架构 (*Qwen2/2.5/3*、*Llama*、*Mistral*、*Gemma*、*MoE* 等)，保持正确性 (逐 *token* 对拍 *HF*)。

1 核心洞察

大部分现代 LLM 的差异只是“配置旋钮”，不是“结构”。适配多模型的关键 = 把硬编码的 transformer block 参数化 + 给真正的结构差异留插件口。

1.1 共享（不变核心）

embedding → $N \times$ decoder block → final norm → LM head；因果自注意力 + KV cache；PagedAttention；前缀缓存。

1.2 可变（要抽象的点）

维度	常见取值
Norm	RMSNorm (Qwen/Llama/Mistral) vs LayerNorm (GPT-2/Phi)
Attention	MHA / GQA / MQA
位置编码	RoPE (θ 可配) / ALiBi / 学习位置 (GPT-2) / 无
MLP	SwiGLU (Qwen/Llama) / GELU (GPT-2) / GeGLU / ReLU
偏置	q/k/v 有无 bias (Qwen2 有、Llama 没有)
结构	pre-norm (主流) / post-norm；串行 attn+MLP / 并行
大结构	MoE (专家路由) / 混合 SSM (Mamba) / MLA (DeepSeek) / 视觉

Qwen2/2.5/3、*Llama*、*Mistral*、*Gemma*、*Phi* 等约 80% 现代模型只是旋钮组合——不需要每模型写一个 forward。

2 架构描述 ArchSpec（配置驱动核心）

```

struct ArchSpec {
    enum class Norm { RMS, LayerNorm };
    enum class PE { RoPE, ALiBi, Learned, None };
    enum class MLP { SwiGLU, GELU, GeGLU, ReLU };
    Norm norm; PE pe; MLP mlp;
    bool qkv_bias = false; // Qwen2 有, Llama 无
    bool pre_norm = true, parallel_attn_mlp = false;
    int hidden, n_heads, n_kv_heads, head_dim, n_layers, vocab;
    float rope_theta = 10000.f, rms_eps = 1e-6f, layer_norm_eps = 1e-5f;
    bool untied_lm_head = false;
    int num_experts = 0, moe_top_k = 0; // MoE
};

```

由 config.json 的 architectures 字段 + 模型参数解析而来。

3 通用 transformer block（一个 forward，多个旋钮）

把 run_layers 层循环参数化——同一套代码按 ArchSpec 分支：

```

void forward_block(Tensor& h, const ArchSpec& arch, const LayerW& w,
                  KV& kv, PosEnc& pos, int layer) {
    h = norm(h, w.norm, arch.norm); // RMS / LayerNorm
    auto [q,k,v] = proj(h, w, arch.qkv_bias); // 同 kernel, GQA
    rope(q, k, pos, arch.rope_theta); // 或 ALiBi / 无
    attn = paged_attention(q, kv, arch); // MHA/GQA 同 kernel
    h = h + attn_o(attn, w); // 残差
    h = h + mlp(norm(h), w, arch.mlp); // SwiGLU / GELU / ...
}

```

换模型 = 换 ArchSpec + 权重映射表，不改 forward。

4 结构差异留插件口 IModelArchitecture

MoE 专家路由、Mamba 混合、MLA、视觉等大结构差异不适合塞旋钮——用接口 + 注册表：

```

class IModelArchitecture {
public:
    virtual const ArchSpec& spec() const = 0;
    virtual void load_weights(SafetensorsLoader&, EngineLayerW&, const ArchSpec&) = 0;
    virtual Tensor forward_layers(Tensor h, const Weights&, KV&, int start_pos, bool write_kv) = 0;
};
NANOINFER_REGISTER_BACKEND(IModelArchitecture, "Qwen2ForCausalLM", Qwen2Arch);
NANOINFER_REGISTER_BACKEND(IModelArchitecture, "LlamaForCausalLM", LlamaArch);

```

- **Qwen2Arch / LlamaArch / MistralArch**：都指向“通用 block + ArchSpec + 权重映射”（配置驱动，几乎零新代码）。

- **Qwen3Arch / DeepSeekArch (MoE)**: 自己的 `forward_layers` (专家路由替换 block 内 MLP)。

5 权重映射（真实工作量最大的一环）

不同模型 tensor 名不同：

```
Qwen2: model.layers.{i}.self_attn.{q,k,v,o}_proj.weight (+bias)
Llama: 同结构但无 bias
Mistral: 同 Llama
```

把 `load_w("model.layers."+ns+".self_attn.q_proj.weight")` 的硬编码改为按架构查映射表，复用现有 `EngineLayerW` 和 `SafetensorsLoader`。

6 Tokenizer 抽象

Qwen/Llama/Mistral 用字节级 BPE(引擎已有);部分模型用 SentencePiece。抽象 `ITokenizer` (encode/decode)，提供 BPE + SP 两个实现。

7 与现有插件化架构的关系

```
model/ ArchSpec + IModelArchitecture 注册表 + 权重映射
engine/ run_layers -> 查注册表调 arch->forward_layers (Qwen2.5 为 reference)
backend/ (已有) GEMM / Attention 插件
tokenizer/ ITokenizer
```

正确性安全网：Qwen2.5 作为 reference arch 保留；每个新 arch 必须过同一套 HF 对拍（“Paris”逐 token 一致 + 增量 == 全量）。

8 扩展路径（由易到难）

阶段	新增	工作量
P-A	重构 <code>run_layers</code> → ArchSpec 通用 block，中（核心重构） Qwen2.5 作 reference	
P-B	配置级加入 Qwen2 / Llama / Mistral (config + 权重映射表)	小（每模型只加映射表）
P-C	Qwen3 (thinking 模式)、MoE 架构	中
P-D	混合 SSM (Mamba)、MLA (DeepSeek)、VL 多模态	大

一句话：“通用 block + ArchSpec 旋钮”覆盖约 80% 模型，`IModelArchitecture` 插件口兜住 MoE/混合/多模态结构差异，权重映射表是每模型的唯一实质工作量。